

CS 1411/2411 Final Project

Madison Davis, Alisona Le, Aarna Pal-Yadav, Christine Tian
School of Engineering & Applied Sciences
Harvard University
Cambridge, Massachusetts

Arya Tschand, David Brooks
School Of Engineering And Applied Sciences
Harvard University
Cambridge, Massachusetts

I. INTRODUCTION

For this project, we implement the Tagged GEometric (TAGE) branch predictor and propose a new design based on it. For our first checkpoint, we reproduce the original TAGE implementation that André Seznec and Pierre Michaud proposed in their 2006 paper “A case for (partially) TAGged GEometric history length branch prediction” [1] in our own system using Pin tools. Next, we evaluate this reconstructed TAGE predictor across two benchmarks (libquantum and dealIII) relative to alternative branch predictors, namely a 1-bit counter, 2-bit saturated counter, and 2-bit saturated counter augmented with a hashed history register table (HHRT). To offer a comprehensive analysis, we also sweep across several TAGE globally-defined parameters, using Intel Pin tools to instrument execution and gather branch traces through automated scripts.

From previous studies in class, we know that modern conditional branch predictors rely on using different history lengths to capture both looping behavior and more complex branch patterns. Simpler predictors often focus on either short histories or fixed-length global histories, which limits their ability to generalize across different workloads. As a result, there is a trade-off between capturing recent behavior and identifying long-range correlations in control flow.

In 2006, Seznec and Michaud address this problem by proposing the TAGE branch predictor. TAGE maintains multiple predictor tables indexed by histories of geometrically increasing lengths, allowing it to adapt to short-term and long-term patterns. By using (partially) tagged entries and selecting predictions based on matching history lengths, TAGE reduces aliasing and improves prediction accuracy compared to earlier designs [1].

In addition to implementing and evaluating the original TAGE design, we propose modifications to the predictor. Our goal is to better understand how different design choices (such as history length selection, tag-bits, and table organization) affect overall prediction performance. Through this work, we aim to evaluate the strengths of TAGE and explore novel opportunities for improving its accuracy and efficiency.

II. IMPLEMENTATION DETAILS

TAGE is a modern implementation of a branch predictor. A K -component TAGE encompasses one base predictor table T_0 and K tagged predictor tables T_i for all $i \in [1, K]$. We assume each table has the same number of rows, 2^N where N is the bit-length of the index hash. Each tagged predictor table T_i incorporates incrementing lengths of history L_i from an ℓ -bit global history register (GHR) that stores, for each of the last ℓ branches, whether it was taken (1) or not taken (0). These history lengths follow a geometric series:

$$L_i = \lfloor R^{i-1} \cdot L_1 + \beta \rfloor$$

Where β is some constant. For example, if $R = 2, \beta = 0.5$, and $L_1 = 2$, then the history lengths of the base predictor table and tagged predictor tables are:

$$\{L_0 = 0, L_1 = 2, L_2 = 4, L_3 = 8, L_4 = 16, L_5 = 32, L_6 = 64, \dots\}$$

The chosen series is arbitrary and can be simplified as:

$$L_i = R^{i-1} \cdot L_1$$

At prediction time, the base predictor table and all tagged predictor tables are accessed simultaneously. The base predictor provides a default prediction, while a tagged component provides a prediction only on a tag match. The final chosen prediction is from the hit tagged predictor table with the longest history, or from the base predictor if no tagged predictor table is hit.

A. Base Predictor Table (T_0)

Each of the 2^N rows of the T_0 base predictor table contains a 2-bit saturating prediction counter, denoted as `pred`. We use the 2-bit saturating counter Autonomata A2 proposed by [4].

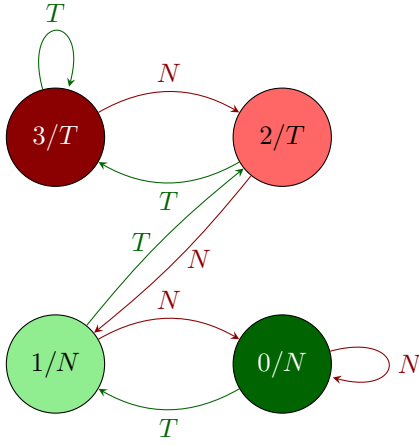


Fig. 1. 2-bit saturating branch predictor (Autonoma A2): states 0–1 predict taken (green) and 2–3 predict not taken (red), with dark/light indicating strong (requires 2 mispredictions to break change prediction)/weak confidence; transitions are labeled by actual outcomes (T/N) and saturate at the extremes.

B. Tagged Predictor Table (T_i)

Each of the 2^N rows of the K tagged predictor tables T_i contains three values:

- 1) a tag
- 2) a 2-bit saturating prediction counter, pred
- 3) a 2-bit saturating usefulness counter, u .

For both saturating counters, the minimum value is 0 and the maximum is 3. When an entry is freshly allocated, we set the following values (the latter pred refers to weakly taken or not taken, respectively):

$$u = 0, \quad \text{pred} = 1 \text{ or } 2$$

C. Index Hashing

The program counter (PC) and GHR are used for indexing into the predictor tables via hashing. If the resulting hash is N bits, then each table has 2^N rows.

Typically, we assign N to be much smaller than the length of the GHR ℓ because the number of entries increases exponentially in N . Therefore, we must compress the ℓ -bit history into N bits to match the index width. This is achieved by a *folding* process, in which we divide the GHR into $\lfloor \frac{\ell}{N} \rfloor$ N -bit segments that we later combine using a bitwise XOR operation, denoted by $\oplus$. Formally,

$$\text{index} = \text{pc}[0 : N - 1] \oplus \bigoplus_{j=0}^{\lceil \ell/N \rceil - 1} \text{GHR}[jN : (j+1)N - 1]$$

As an example, consider $K = 4$ tagged predictor tables, each with $2^{10} = 1024$ rows, indexed with $L_0 = 10$, $L_1 = 20$, $L_2 = 40$, and $L_3 = 80$ history bits from an ($\ell = 80$)-bit GHR. For table that uses $L_2 = 40$ history bits, the index is:

$$\begin{aligned} \text{index} = & \text{pc}[0 : 9] \oplus \text{GHR}[0 : 9] \oplus \text{GHR}[10 : 19] \\ & \oplus \text{GHR}[20 : 29] \oplus \text{GHR}[30 : 39] \end{aligned}$$

D. Tag Hashing

The tag is also computed using a hash of the PC and GHR. If the tag has M bits, the hash output must be M bits to match.

This hash incorporates two circular shift registers (CSRs) as an alternative to the bitwise-XOR folding process used in index hashing. CSRs are a compact data structure that compress the long GHR into a fixed number of bits by rotating in the newest history bit at each update step. We use one CSR of width M and the other of width $M - 1$ to improve the quality of the hash. Formally,

$$\text{tag} = \text{pc}[0 : M - 1] \oplus \text{CSR}_M \oplus \text{CSR}_{M-1}$$

For example, if $M = 8$:

$$\text{tag} = \text{pc}[0 : 7] \oplus \text{CSR}_8 \oplus \text{CSR}_7$$

E. Prediction Update

Regardless of the correctness of our prediction, the prediction counter is updated as follows:

- 1) Increment pred (capped at 3) if the branch is taken,
- 2) Decrement pred (floored at 0) if the branch is not taken.

If TAGE correctly predicts whether the branch was taken, we perform the following step:

- 1) Update the usefulness counter u_i if $\text{pred}(T_i)$ is correct and $\text{pred}(T_{alt})$ is incorrect:

$$u_i \leftarrow \min(u_i + 1, 3)$$

where T_i is our chosen matching table and T_{alt} is the matching table with the next-longest history length:

$$T_{alt} = T_j \quad \text{where} \quad j = \max_{k \in \{0, \dots, i-1\}} (\text{TagMatch}(T_k))$$

If TAGE incorrectly predicts whether the branch was taken, we perform the following steps:

- 1) Update the usefulness counter u_i if $\text{pred}(T_i)$ is incorrect and $\text{pred}(T_{alt})$ is correct:

$$u_i \leftarrow \max(u_i - 1, 0)$$

This outcome indicates that the longer history predictor was less useful than expected.

- 2) Attempt an allocation in longer history tables. More specifically, if T_i is not the longest history table, attempt to allocate an entry in a table T_h with longer history ($h > i$) according to the following rules:

- a) Allocate only one entry at most.
- b) If an entry with $u_h = 0$ exists, evict it and allocate a new entry.
- c) If multiple tables T_h and T_j ($h < j$) satisfy $u_h = u_j = 0$, choose T_h with higher probability prob .
- d) If no allocation is possible (all $u \neq 0$), decrement usefulness counters of candidate entries:

$$u \leftarrow u - 1$$

III. DIFFERENCES FROM THE ORIGINAL TAGE DESIGN

Our implementation of the TAGE branch predictor differs from the original design proposed by Seznec and Michaud in several ways.

First, we define N_{TABLES} as the total number of prediction tables, including the base predictor T_0 , as opposed to just the number of tagged prediction tables.

Second, the original paper uses a 3-bit signed counter for the prediction finite state machine (FSM). In contrast, we use a 2-bit unsigned saturating counter (Automata A2 from [4]). While this design still incorporates hysteresis, it provides less granularity than the original 3-bit scheme.

Finally, we do not maintain a dedicated circular shift register (CSR) structure for tag hashing. Instead, we dynamically construct the equivalent CSR values when computing the tag hash by folding appropriate segments of the global history register (GHR). This simplifies the implementation at the cost of recomputing during prediction.

IV. BENCHMARKS

We look at three main benchmarks:

A. *libquantum_O3*

This benchmark simulates a quantum computer executing Shor’s factorization algorithm using large one-dimensional arrays, producing highly regular and predictable branch patterns. As a result, TAGE is expected to achieve near-perfect accuracy, since its base predictor and short-history tagged tables can quickly learn and saturate confidence counters for these repeating behaviors.

B. *dealII_O3*

This benchmark represents a C++ finite element analysis library with complex, deeply nested control flow. TAGE should excel here due to its geometrically increasing history lengths, which capture long-range correlations common in such workloads, while its tagging mechanism mitigates aliasing. However, the large instruction footprint may increase pressure on predictor tables and require more frequent aging of useful bits.

C. *Comparison*

Overall, while TAGE performs near optimally on the highly regular patterns of *libquantum*, its advanced features—long histories and tagging—are especially critical for handling the complexity, scale, and aliasing challenges present in *dealII*.

V. COMPARING TAGE TO OTHER BRANCH PREDICTORS

We evaluate the performance of our TAGE implementation against several baseline branch predictors:

- 1) a 1-bit predictor,
- 2) a 2-bit saturating counter predictor,
- 3) a 2-bit saturating counter HHRT-based predictor.

The 1-bit predictor and 2-bit predictor are set up as follows: the program counter (PC) is masked to serve as an index into a branch prediction buffer (BPB), where each entry is either a 1-bit or 2-bit saturated counter, respectively. The 2-bit HHRT predictor uses a hashed program counter (PC) to index into a hashed history register table (HHRT), which provides a branch history register (BHR). This BHR is then used to index into a pattern history table (PHT) containing 2-bit saturating counters.

All experiments are conducted using our group’s default parameter configurations defined by Programming Assignment 2’s optimal sweeping values. Namely, for the 1-bit and 2-bit saturated counter, we use a branch prediction buffer (BPB) of 1024 entries. For the 2-bit HHRT saturated counter, we used an HHRT of 512 entries and a per-PHT table of 512 entries. For TAGE, we used a geometric series defined by $R = 2$ and $L_1 = 12$ (referencing the first tagged predictor table); 10 total tables, including the base predictor table; the lower 12 bits of the PC used for hashing; 4096 entries per table (2^{12}); 10 tag bits per entry; a probability ratio of 3 : 1 when choosing between two higher tagged predictor tables for entry eviction; and a global history of 4096 bits. The results are shown below. We see that TAGE successfully beats out the other branch predictors across the two benchmarks.

TABLE I
BRANCH PREDICTOR MISS RATES (%) USING DEFAULT PARAMETERS

Predictor	libquantum	dealII
TAGE	0.0735	1.1784
2-bit HHRT	0.4535	3.6064
2-bit	6.7500	12.4549
1-bit	8.7750	17.7960

VI. PARAMETER SWEEPS

The aforementioned simulation parameters for TAGE were chosen by trial-and-error. To provide a more robust and comprehensive analysis, we perform a number of sweeps to determine an optimal configuration for our TAGE predictor. For values that were not tuned in each sweep, they were set to the default values presented in Table II.

TABLE II
PARAMETER DEFAULT VALUES

Parameter	Default Value
L1	4
R	2
NTABLES	6
PC_BITS	10
NENTRIES	1024
TAG_BITS	8
NHIST	128
PROB	2

A. *R and L₁*

We tuned the values R and L_1 , which correspond to the common ratio and initial value of the geometric series of

history lengths of the tagged tables, respectively. For the R sweep, we fixed $L_1 = 4$ and tested the values $[2, 3, 4, 5]$. The results of the sweeps are as follows:

TABLE III
BRANCH PREDICTOR MISS RATES (%) FOR SWEEP OVER R VALUES

R	libquantum	dealII
2	0.09052	1.4956
3	0.0581	1.3350
4	0.0729	1.4594
5	0.09046	1.5566

We had originally hypothesized that increasing R would decrease both miss rates as it would by proxy increase the amount of history that would be used to make a decision. However, we noticed an interesting pattern where starting at $R = 4$, miss rates would actually start to increase after a period of decreasing. We hypothesize that this is because the histories are limited by the global history length cap at 128 bits. With $R = 4, 5$, many of the tables all get capped at a history of 128, effectively making the tables redundant. Although this should also be true for $R = 3$, perhaps the particular history lengths it produces aren't as prone to noise compared to $R = 4, 5$ because of the shorter history lengths.

For the L_1 sweep, we fixed $R = 2$ and tested the values $[4, 6, 8, 10, 12]$ with the rest of the default parameters.

TABLE IV
BRANCH PREDICTOR MISS RATES (%) FOR SWEEP OVER L_1 VALUES

L_1	libquantum	dealII
4	0.0905	1.4956
6	0.0633	1.4213
8	0.0547	1.2912
10	0.0538	1.2973
12	0.0517	1.3046

We see a similar phenomenon as in the R sweep where increasing L_1 beyond $L_1 = 8$ degrades performance for the dealII benchmark. Interestingly, this trend is not seen in libquantum. We hypothesize that libquantum's workload is better suited for the history lengths produced by $L_1 = 10, 12$.

B. Tag Bits

For the tag bits sweep, we tested the values $[8, 10, 12]$. Compared to the L_1 and R sweeps, there is less of a difference in performance across these values. With a larger tag, specific branches can be identified at a finer granularity that leads to less aliasing issues. With this, branches are more likely to get a more accurate snapshot of their recent actions, essentially making the prediction more well informed. This would thus lead to lower miss rates.

TABLE V
BRANCH PREDICTOR MISS RATES (%) FOR SWEEP OVER TAG BITS

Tag bits	libquantum	dealII
8	0.0919	1.4956
10	0.1004	1.4845
12	0.0854	1.4695

C. Probability of Allocating Shortest Table

TABLE VI
BRANCH PREDICTOR MISS RATES (%) FOR SWEEP OVER PROBABILITIES

Prob	libquantum	dealII
2	0.0917	1.4956
3	0.0925	1.4961
4	0.0940	1.4963

An important part of the TAGE predictor's efficiency is its entry allocation policy during mispredictions. When the matched component T_i makes an incorrect prediction, the predictor attempts to allocate a new entry in a table with a longer history, T_h , where $h > i$. If multiple tables in the range $T_{i+1} \dots T_n$ contain entries with a usefulness bit of $u = 0$, a probabilistic policy selects a target, biasing toward the shortest available history with probability $\text{Pr}(\text{shortest}) = \frac{\text{prob}-1}{\text{prob}}$. This conserves resources by preventing long-history tables from being saturated with simple patterns. However, increasing prob (e.g., from 2 to 4) slightly increases miss rates due to increased aliasing in shorter tables as they become overcrowded. This tradeoff particularly impacts workloads with complex control flow that depend on long-range correlations.

D. PC Bits and Number of Entries

TABLE VII
BRANCH PREDICTOR MISS RATES (%) FOR VARYING PC INDEX BITS AND NUMBER OF ENTRIES IN TABLE SIZES.

Configuration	libquantum	dealII
pc=4, n=16	0.3042	4.4194
pc=8, n=256	0.1769	1.6810
pc=10, n=1024	0.0914	1.4955
pc=12, n=4096	0.0907	1.4091
pc=14, n=16384	0.1706	1.3533
pc=16, n=65536	0.1304	1.3232

We see that as we increase the number of PC bits and number of entries, the miss rate generally decreases for the dealII benchmark and exhibits a "U-shape" behavior for the libquantum benchmark.

This difference in behavior comes from what occurs in each benchmark. libquantum may exist in a setting where both sides of a large PC count and number of entries matter, since it has a repetitive set of static PCs with strong patterns. dealII may also have a larger set of diverse branch PCs.

We hypothesize that this overall pattern reflects a benchmark-dependent tradeoff between aliasing reduction and training density. Increasing hashed PC bits or the number of entries reduces aliasing by decreasing collisions in the predictor index. But, this also spreads branch instances across a larger set of entries, reducing the number of updates per entry and thus lowering training density. As a result, many more entries are infrequently accessed and become less well trained. Additionally, useful correlations between branches may be diluted as related patterns are distributed across a larger index

space, reducing constructive reinforcement. This sparsity, combined with increased sensitivity to noise in less frequently updated entries, results in a slight increase in miss rates at higher PC widths. Overall, our results demonstrate a tradeoff between reducing aliasing and maintaining sufficient training density, with intermediate configurations of PC and number of entries providing a good balance for both benchmarks.

E. Number of Predictor Tables

TABLE VIII
BRANCH PREDICTOR MISS RATES (%) VS. NUMBER OF TAGE TABLES.

Table Ct.	libquantum	dealII
4	0.6242	3.0258
6	0.0914	1.4956
8	0.0559	1.3067
12	0.0597	1.4083

We see that as we increase the number of tables, our miss rate initially drops. For instance, from table size 4 to 6 we see an extreme drop, and then again from 6 to 8. We see that from 8 to 12 that the miss rate slightly increases. This can be due to increased competition among excess tables (for example, several tables can be chosen as candidates at a given time for prediction or eviction, making it more difficult to decipher which provides more relatively-important information), which can produce noise. Overall, these results hint that a moderate number of tables are a good balance between long-range correlation and maintaining efficiency.

VII. PROPOSED EXTENSIONS

To further improve the baseline TAGE predictor, we explore several potential extensions aimed at increasing prediction accuracy and robustness. Due to time constraints, only a subset of these ideas may be fully implemented and evaluated. The goal is to have at least one of these novel ideas produce better results (lower miss rates) when compared to the initial TAGE implementation on the same parameters and benchmarks.

A. Confidence-Based Table Selection

The baseline TAGE predictor selects the prediction from the table with the longest matching history. We propose augmenting this decision with a confidence mechanism. Specifically, we compare the predictions from multiple tables corresponding to shorter histories. If multiple tables agree on the prediction, we assign higher confidence and increase the likelihood of selecting the longer-history predictor. Conversely, disagreement across tables may indicate lower confidence.

B. Mitigating Long-History Overfitting

Long-history tables may be prone to overfitting. To address this, we propose a heuristic selection mechanism: if the selected table produces a weak prediction (counter values 1 or 2) while an alternative table provides a strong prediction (0 or 3), we probabilistically favor the alternative table. The selection probability is proportional to the ratio of the alternative history length to the target history length.

C. History Weighting in Hashing

In the current implementation, the history compression function treats all bits in the global history register (GHR) equally. We propose biasing the hash toward more recent history by assigning greater weight to recent bits. For example, when folding history segments via XOR, the most recent segment may be incorporated multiple times to increase its influence. This may better capture correlations amongst branches.

D. PC-Based Bias Predictor

One limitation of TAGE is that highly biased branches (e.g., loop exits) may map to different entries due to history-based indexing, reducing prediction stability. To mitigate this, we propose a PC-indexed bias predictor table that depends only on lower bits of the program counter. Unlike the base predictor, this table can override the selected TAGE prediction when it exhibits high confidence, thereby stabilizing predictions for strongly biased branches.

E. Adaptive History Length Selection

We propose tracking the effectiveness of different history lengths on a per-branch basis. By associating prediction accuracy with specific program counters, the predictor can dynamically favor history lengths that have historically yielded higher accuracy, potentially overriding the default longest-history selection policy.

F. Alternative Counter Designs

The standard 2-bit saturating counter may be replaced with alternative designs, such as asymmetric counters biased toward taken or not-taken outcomes. These modifications may better capture skewed branch behaviors.

G. Alternative Replacement Policies

Finally, instead of relying solely on usefulness counters for entry replacement, we consider incorporating recency-based policies. For example, entries that have not been accessed recently may be preferentially evicted, similar to LRU strategies.

REFERENCES

- [1] André Seznec and P. Michaud, "A case for (partially) tagged geometric history length branch prediction," HAL (Le Centre pour la Communication Scientifique Directe), Feb. 2006.
- [2] André Seznec, "TAGE-SC-L Branch Predictors," JILP - Championship Branch Prediction, Jun 2014, Minneapolis, United States.
- [3] P. Michaud, "A PPM-like, tag-based branch predictor," HAL (Le Centre pour la Communication Scientifique Directe), Apr. 2005.
- [4] T.-Y. Yeh and Y. N. Patt, "Two-level adaptive training branch prediction," Sep. 1991, doi: <https://doi.org/10.1145/123465.123475>.
- [5] Davis, Madison and Le, Alisona and Pal-Yadav, Aarna and Tian, Christine, "CS 1411 Final Project," <https://github.com/Madison-Davis/Courses-CS1411-FinalProject>